1. Declare an integer variable called likes and a pointer variable called ptrLikes; assign likes a value, point ptrLikes to likes, and print both the value and the address stored in ptrLikes.

```c
#include <stdio.h>

int main() {
    // Declare an integer variable
    int likes = 150;

    // Declare a pointer and point it to likes
    int *ptrLikes = &likes;

    // Print the value of likes
    printf("Value of likes: %d\n", likes);

    // Print the value using the pointer
    printf("Value using ptrLikes: %d\n", *ptrLikes);

    // Print the address stored in the pointer
    printf("Address stored in ptrLikes: %p\n", (void *)ptrLikes);

    return 0;
}
```

2. Write a function swapPlaylistCounts(int *a, int *b) that swaps the number of songs in two Spotify playlists using pointers, then call the function in main and print the swapped values.

```c
    #include <stdio.h>


// Function to swap two integers using pointers
void swapPlaylistCounts(int *a, int *b) {
    int temp;

    temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int playlist1 = 25;
    int playlist2 = 40;

    printf("Before Swapping:\n");
    printf("Playlist 1 Songs = %d\n", playlist1);
    printf("Playlist 2 Songs = %d\n", playlist2);

    // Call the swap function
    swapPlaylistCounts(&playlist1, &playlist2);
```

```
    printf("\nAfter Swapping:\n");
    printf("Playlist 1 Songs = %d\n", playlist1);
    printf("Playlist 2 Songs = %d\n", playlist2);

    return 0;
}
```

**3. Given an array of 5 order amounts (e.g., Zomato orders), use a pointer to iterate through the array and print each amount along with its memory address.

Hint: Use pointer arithmetic to move to the next element.**

```
#include <stdio.h>

int main() {
    // Array of 5 order amounts
    int orders[5] = {250, 320, 180, 450, 300};

    // Pointer pointing to the first element of the array
    int *ptr = orders;

    printf("Order Amounts and Their Memory Addresses:\n");

    // Iterate through the array using pointer arithmetic
    for (int i = 0; i < 5; i++) {
        printf("Order %d: Amount = %d, Address = %p\n",
               i + 1, *(ptr + i), (void *)(ptr + i));
    }

    return 0;
}
```

4. Create a function incrementFollowers(int *followers, int n) that increases each follower count in an array (representing Instagram followers for 5 friends) by 100 using pointer arithmetic, then print the updated counts.

```
#include <stdio.h>

// Function to increase each follower count by 100
void incrementFollowers(int *followers, int n) {
    for (int i = 0; i < n; i++) {
        *(followers + i) = *(followers + i) + 100;
    }
}

int main() {
    int followers[5] = {1200, 850, 2300, 1750, 950};

    printf("Follower Counts Before Increment:\n");
```

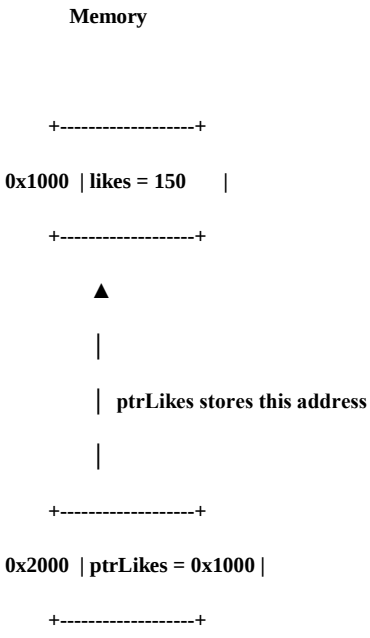
```
    for (int i = 0; i < 5; i++) {
        printf("Friend %d: %d\n", i + 1, followers[i]);
    }

    // Call the function
    incrementFollowers(followers, 5);

    printf("\nFollower Counts After Increment:\n");
    for (int i = 0; i < 5; i++) {
        printf("Friend %d: %d\n", i + 1, followers[i]);
    }

    return 0;
}
```

5. Draw a simple memory diagram (on paper or using any drawing tool) showing how a pointer references the address of a variable, using the variables from your likes and ptrLikes example; label the variable, pointer, and address.



Code:

```
#include <stdio.h>

int main() {
    int likes = 150;
    int *ptrLikes = &likes;

    printf("likes = %d\n", likes);
    printf("*ptrLikes = %d\n", *ptrLikes);
    printf("Address of likes = %p\n", (void *)&likes);
    printf("Value stored in ptrLikes = %p\n", (void *)ptrLikes);
```

```
    return 0;
}
```